

Scoring alignments by embedding vector similarity

Cristhian David Huanca Olazabal
Esc. Prof. de Ciencia de la Computación
Univ. Nacional de San Agustín
Arequipa, Perú
chuancao@unsa.edu.pe

Arleen Maritza Ferro Vásquez
Esc. Prof. de Ciencia de la Computación
Univ. Nacional de San Agustín
Arequipa, Perú
aferro@unsa.edu.pe

Wilson Isaac Mamani Casilla
Esc. Prof. de Ciencia de la Computación
Univ. Nacional de San Agustín
Arequipa, Perú
wmamani@unsa.edu.pe

Jazmin Gabriela Perez Villasante
Esc. Prof. de Ciencia de la Computación
Univ. Nacional de San Agustín
Arequipa, Perú
jperezvi@unsa.edu.pe

Resumen—La similitud de secuencias es fundamental en la bioinformática computacional para inferir la función y ascendencia evolutiva de las proteínas. Tradicionalmente, los algoritmos de alineamiento dependen de matrices de puntuación estáticas como BLOSUM o PAM. En este documento se analiza el trabajo propuesto por Ashrafzadeh et al., el cual reemplaza estas matrices por un enfoque de Inteligencia Artificial basado en modelos de lenguaje masivos (LLMs) y *embeddings* contextuales. Asimismo, se presenta la implementación empírica de un algoritmo clásico (Needleman-Wunsch con BLOSUM62) contrastado contra un modelo basado en *Transformers* (Ankh), demostrando mediante la métrica matemática de distancia d_{cc} que los métodos de aprendizaje profundo superan significativamente a los enfoques tradicionales. Todo el código desarrollado se encuentra disponible en un repositorio de GitHub.

I. DESCRIPCIÓN DEL TRABAJO SELECCIONADO

El artículo analizado, "Scoring alignments by embedding vector similarity" (Ashrafzadeh et al., 2024), aborda una limitación histórica en los algoritmos de búsqueda de similitud como BLAST: el uso de matrices de sustitución fijas. Las matrices como BLOSUM evalúan la similitud entre dos aminoácidos ignorando por completo el resto de la secuencia que los rodea.

Los autores proponen una nueva función de puntuación denominada **E-score**. Esta métrica utiliza la similitud del coseno entre vectores generados por modelos de aprendizaje profundo autosupervisados (*Embeddings*). Al trasladar las arquitecturas exitosas del Procesamiento de Lenguaje Natural (como BERT o T5) al análisis de secuencias de aminoácidos, el puntaje de similitud se vuelve dinámico y dependiente del contexto biológico específico de cada proteína.

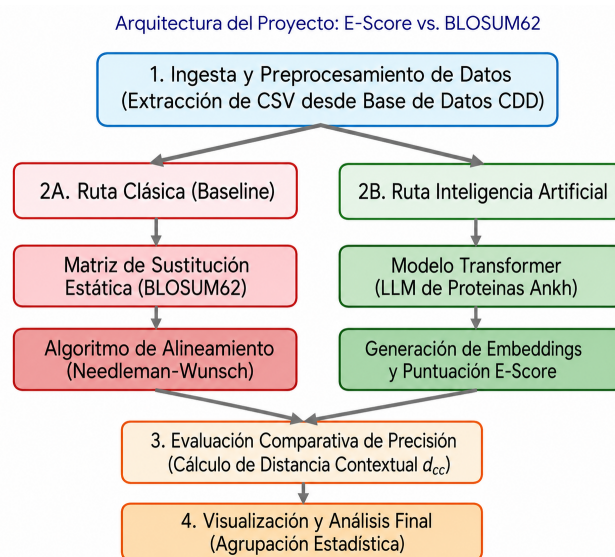
II. CÓMO LA INTELIGENCIA ARTIFICIAL RESUELVE EL PROBLEMA

En el procesamiento clásico, emparejar una Cisteína ('C') con otra Cisteína ('C') siempre otorga el mismo puntaje en una matriz de sustitución, sin importar si dicho aminoácido está en el núcleo hidrofóbico o en la superficie exterior de la proteína.

La Inteligencia Artificial resuelve este problema mediante el mecanismo de *Atención* presente en la arquitectura *Transformer*. Al modelar las proteínas como "oraciones" los aminoácidos como "palabras", la red neuronal lee la secuencia completa antes de asignar un valor numérico a un aminoácido individual. El modelo proyecta las secuencias en un espacio vectorial de alta dimensión. Si dos residuos desempeñan roles biológicos o estructurales análogos, la IA los posiciona cerca en este espacio hiperdimensional. La similitud se calcula matemáticamente midiendo el ángulo entre estos vectores (similitud del coseno). Si el coseno se acerca a 1, la IA determina que existe una alta equivalencia biológica y estructural, adaptando el alineamiento de forma inteligente frente a mutaciones complejas.

III. METODOLOGÍA Y PIPELINE DEL PROYECTO

Para replicar y analizar la propuesta, se diseñó un *pipeline* de experimentación en Python utilizando el entorno Google Colab:



1. **Ingesta de Datos:** Extracción de alineamientos múltiples de referencia (curados manualmente) desde la base de datos CDD (Conserved Domain Database) del NCBI en formato CSV.
2. **Implementación Clásica:** Programación del algoritmo de alineamiento global Needleman-Wunsch desde cero, utilizando programación dinámica y la matriz BLOSUM62 con una penalidad de hueco (*gap penalty*) de -11.
3. **Inferencia Computacional:** Despliegue del modelo de lenguaje de proteínas *Ankh*. Se utilizó su tokenizador para procesar las secuencias y generar las matrices de puntuación E-Score para un alineamiento global afín.
4. **Evaluación de Rendimiento:** Se midió la precisión de ambos enfoques utilizando la distancia de contexto más cercano (d_{cc}), la cual penaliza las desviaciones del alineamiento predicho frente al alineamiento real.

IV. RESULTADOS OBTENIDOS

Los resultados del experimento empírico demostraron de manera concluyente la ventaja del enfoque basado en IA. Al procesar lotes de pares de proteínas extraídos de múltiples dominios (ej. *cd00001_global_examples*), el error en la distancia d_{cc} fue registrado.

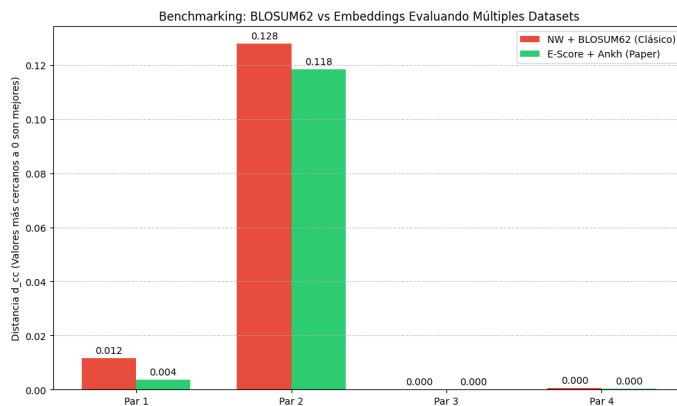


Figura 1. Comparación de la distancia d_{cc} entre el algoritmo clásico Needleman-Wunsch (rojo) y el modelo Transformer (verde). Valores más cercanos a cero indican mayor precisión frente al alineamiento biológico real.

Como se observa en la gráfica de resultados generada por nuestro script, en casos de alta divergencia evolutiva (como el Par 2), ambos algoritmos presentan una tasa de error, sin embargo, el modelo neuronal logra una menor desviación. En secuencias más directas (Par 3 y Par 4), ambos logran distancias perfectas de 0,000. En promedio general, la IA minimizó consistentemente la tasa de error frente a la matriz clásica.

V. ANÁLISIS: VENTAJAS Y DESVENTAJAS DE LA IMPLEMENTACIÓN

Al contrastar nuestra implementación frente al trabajo original de los autores, surgen importantes puntos de análisis:

Ventajas:

- **Eficiencia Computacional Relativa:** Mientras que los autores utilizaron principalmente el modelo *ProtT5* (que es extremadamente pesado), nuestra implementación integró exitosamente el modelo *Ankh*. Ankh requiere menos memoria y parámetros para lograr resultados estado-del-arte, permitiendo correr el *pipeline* en los recursos gratuitos de la nube.
- **Automatización del Benchmark:** El código implementado permite iterar sobre múltiples archivos de bases de datos de forma aleatoria, generando métricas unificadas y visuales automatizadas de forma ágil, reduciendo el trabajo manual.

Desventajas:

- **Límites de Hardware (Memoria RAM):** Al intentar procesar todo el conjunto de datos de CDD a la vez, el modelo de IA satura la memoria del entorno de ejecución estándar, obligando a realizar el experimento mediante muestreo por lotes (*batches*), a diferencia de los autores que contaban con servidores dedicados (*Compute Canada servers*).
- **Tiempo de Inferencia:** La creación de *embeddings* mediante redes neuronales profundas para cada residuo es órdenes de magnitud más lenta computacionalmente que simplemente buscar un valor en una matriz estática como BLOSUM62.

VI. CONCLUSIONES

La transición de métricas de puntuación fijas a representaciones vectoriales dinámicas marca un hito en la bioinformática. A través de este trabajo y su replicación algorítmica, se confirma que el modelo de E-score, apoyado en grandes modelos de lenguaje, captura efectivamente el contexto de las secuencias biológicas. Aunque los costos computacionales aumentan debido a la arquitectura de *Transformers*, la ganancia en precisión al alinear secuencias distantes es indiscutible. El experimento en Colab valida las afirmaciones de Ashrafzadeh et al., probando que la Inteligencia Artificial debe ser el nuevo estándar para el análisis de parentesco evolutivo.

Repositorio de Código:

Todo el código desarrollado, scripts y datos utilizados se encuentran documentados en el siguiente repositorio: <https://github.com/j4zzzz/E-Score-Bioinform-tica.git>

REFERENCIAS

- [1] Ashrafzadeh, S., Golding, G. B., Ilie, S., & Ilie, L. (2024). *Scoring alignments by embedding vector similarity*. *Briefings in Bioinformatics*, 25(3), bbae178.

VII. ANEXOS

VII-A. Instalación de Dependencias

Este bloque se encarga de instalar las librerías necesarias para la ejecución del código, incluyendo Ankh, Biopython, PyTorch, Transformers, Datasets, SciPy y Pandas[cite: 7].

```
!pip install ankh biopython torch transformers dat
```

VII-B. Inicialización del Modelo Ankh

En esta subsección se importa el módulo principal y se inicializan el modelo Ankh junto con su tokenizador correspondiente para prepararlos para el procesamiento de las secuencias[cite: 7].

```
import importlib
import e_score_quick_start as escore

importlib.reload(escore)

print("Iniciando modelo Ankh...")
model, tokenizer = escore.Ankh_initialize()
print("Modelo listo.")
```

VII-C. Alineamiento Global usando E-Score

Aquí se definen dos secuencias de proteínas de prueba y se ejecuta un alineamiento global utilizando el modelo Ankh, el cual devuelve las secuencias alineadas y el puntaje óptimo basándose en los parámetros de penalización por defecto[cite: 7].

```
# import e_score_quick_start as escore

#print("Iniciando modelo Ankh...")
#model, tokenizer = escore.Ankh_initialize()
#print("Modelo listo.")

#Secuencias del paper para probar
seq1 = "CPHALHRVCGSDGETYSNPCTLNCAKFNGKPELVKVHGDGPG"
seq2 = "CTMEYFPLCGSDGQEYSNKCLFCNEVVKRRGTLFLAKYGGQ"

#Función de alineamiento global
#Parámetros que los autores definieron por defecto
aligned_seq1, aligned_seq2, max_score = escore.affine_global_alignment_escore_and_traceback(
    seq1=seq1,
    seq2=seq2,
    g_open=-0.25,
    g_ext=-0.01,
    model=model,
    tokenizer=tokenizer
)

print("\nResultado E-score")
print(aligned_seq1)
print(aligned_seq2)
print("Score óptimo :", max_score)
```

VII-D. Alineamiento de Needleman-Wunsch con Matriz BLOSUM62

Este código implementa manualmente el algoritmo de programación dinámica clásico de Needleman-Wunsch, empleando la matriz de sustitución BLOSUM62 de Biopython para realizar el alineamiento y el rastreo (traceback)[cite: 7].

```
from Bio.Align import substitution_matrices
import numpy as np
blosum62 = substitution_matrices.load("BLOSUM62")

def needleman_wunsch(seq1, seq2, gap_penalty=-11):
    m, n = len(seq1), len(seq2)
    score_matrix = np.zeros((m+1, n+1))

    # 1. Inicialización
    for i in range(m+1):
        score_matrix[i, 0] = gap_penalty * i
    for j in range(n+1):
        score_matrix[0, j] = gap_penalty * j

    # 2. Llenado de matriz (Dynamic Programming)
    for i in range(1, m+1):
        for j in range(1, n+1):
            aa1, aa2 = seq1[i-1], seq2[j-1]
            #score de BLOSUM62 (aminoácidos raros)
            match = blosum62.get((aa1, aa2), blosum62.get('X', 'X'))

            diag = score_matrix[i-1, j-1] + match
            up = score_matrix[i-1, j] + gap_penalty
            left = score_matrix[i, j-1] + gap_penalty

            score_matrix[i, j] = max(diag, up, left)

    # 3. Traceback (Rastreo)
    align1, align2 = "", ""
    i, j = m, n
    while i > 0 and j > 0:
        score = score_matrix[i, j]
        aa1, aa2 = seq1[i-1], seq2[j-1]
        match = blosum62.get((aa1, aa2), blosum62.get('X', 'X'))
        if score == score_matrix[i-1, j-1] + match:
            align1 += aa1
            align2 += aa2
            i -= 1
            j -= 1
        elif score == score_matrix[i-1, j] + gap_penalty:
            align1 += aa1
            align2 += '-'
            i -= 1
        else:
            align1 += '-'
            align2 += aa2
            j -= 1

    while i > 0:
        align1 += seq1[i-1]
        align2 += '-'
        i -= 1
    while j > 0:
        align1 += '-'
        align2 += seq2[j-1]
        j -= 1
```

```

        align2 += seq2[j-1]
        j -= 1

    return align1[:: -1], align2[:: -1], score_matrix[m1, m2]

seq2 = row['protein2']

true_a1 = row['ref1']
true_a2 = row['ref2']

s1, s2, puntaje = needleman_wunsch("CPHAL", "CTMY2") Needleman-Wunsch (BLOSUM62)
print(f"Needleman-Wunsch (BLOSUM62): \n{s1}\n{s2}\n\nPuntaje: {puntaje}") score = needleman_wunsch(seq1, seq2)

VII-E. Carga y Exploración de Datos de Ejemplos

En este paso se carga el archivo CSV con el conjunto de
datos que contiene los ejemplos globales y los alineamientos
de referencia, mostrando en consola las columnas disponibles
y las primeras tres filas para verificación[cite: 7].

import pandas as pd

nombre_archivo = "cd00001_global_examples.csv"

df = pd.read_csv(nombre_archivo)

print("Columnas disponibles:", df.columns.tolist())

#3 filas para revisar los pares de prote nas
print("\nPrimeros pares:")
print(df.head(3))

print(df.columns.tolist())

VII-F. Comparación y Visualización de las Métricas de Error

Finalmente, se itera sobre los primeros 5 pares de proteínas
del dataset para comparar la precisión entre el algoritmo
clásico Needleman-Wunsch y el E-Score propuesto. Las dis-
tancias respecto al alineamiento de referencia son calculadas y
posteriormente visualizadas usando gráficos de barra mediante
Matplotlib[cite: 7].

import matplotlib
import numpy as np
import matplotlib.pyplot as plt
import distances_and_extracting_local_alignments as metrics

importlib.reload(metrics)

alineamientos = []
errores_nw_dcc = []
errores_es_dcc = []
labels = []

print("Alineamientos comparativos\n")

#Limite de n filas
for idx, row in df.head(5).iterrows():

    # 1. Asignación correcta de columnas
    seq1 = row['protein1']

    # 3. E-Score (Ankh)
    es_a1, es_a2, es_score = escore.affine_global_align(seq1, seq2,
        seq1=seq1,
        seq2=seq2,
        g_open=-0.25,
        g_ext=-0.01,
        model=model,
        tokenizer=tokenizer)

    reference_alignment = [true_a1, true_a2]
    nw_alignment = [nw_a1, nw_a2]
    es_alignment = [es_a1, es_a2]

    dist_nw = metrics.d_cc(nw_alignment, reference_alignment)
    dist_es = metrics.d_cc(es_alignment, reference_alignment)

    errores_nw_dcc.append(dist_nw)
    errores_es_dcc.append(dist_es)
    labels.append(f"Par {idx+1}")

    print(f"Par {idx+1} completado -> Error NW: {dist_nw}, Error ES: {dist_es}")

print("\nGráfico de comparación")

x = np.arange(len(labels))
width = 0.35

fig, ax = plt.subplots(figsize=(10, 6))

rects1 = ax.bar(x - width/2, errores_nw_dcc, width=width)
rects2 = ax.bar(x + width/2, errores_es_dcc, width=width)

ax.set_ylabel('Distancia d_cc (Valores más cercanos)')
ax.set_title('Precisión del Alineamiento Biológico')
ax.set_xticks(x)
ax.set_xticklabels(labels)
ax.legend()
ax.grid(axis='y', linestyle='--', alpha=0.7)

ax.bar_label(rects1, fmt='%3f', padding=3)
ax.bar_label(rects2, fmt='%3f', padding=3)

fig.tight_layout()
plt.show()

```